
SISTEME DE OPERARE
EXAMEN FINAL
6 FEBRUARIE 2026

INSTRUCȚIUNI

- Vă rog să vă scrieți numele pe fiecare foaie pe care o predați (inclusiv pe foile cu subiectele de examen pe care le veți returna împreună cu celelalte foi).
- Examenul se ține fără documentație pe masă și fără acces la echipamente electronice (telefon mobil, tabletă, ceas inteligent, etc.).
- Aveți **120 minute** la dispoziție pentru a termina examenul. Dați răspunsuri concise și evitați să pierdeți vremea furnizând detalii irelevante sau care nu sunt solicitate.
- Punctajul maxim este de **100 de puncte**.

SUBIECTE

1. (5 pcte) Adevăr sau fals? Marcați corespunzător fiecare afirmație de mai jos.

- (a) Sincronizarea proceselor presupune excludere mutuală.
- (b) Race conditions nu se pot rezolva corect folosind presupuneri legate de viteza relativă de execuție a proceselor.
- (c) Secțiunea critică afectează inversiunea de priorități.
- (d) Condiția de progres stipulează că accesul unui proces în secțiune critică nu poate fi întârziat indefinit.
- (e) Una dintre condițiile de producere a deadlock-ului presupune sincronizarea proceselor.

2. (15 pcte) Gestiunea memoriei, memorie virtuală.

- (a) (3 pcte) Dați 3 exemple de algoritmi de alocare dinamică a memoriei cu partiții variabile (gestiunea găurilor/holes care apar în asemenea sisteme). Explicați pe scurt cum funcționează cei trei algoritmi aleși.
- (b) (3 pcte) Ce este mecanismul de paginare și care e diferența principală față de alocarea contiguă a memoriei?
- (c) (4 pcte) Cum se transformă adresa virtuală în adresă fizică în *segmentarea pură* (fără paginare)? O diagramă corespunzător adnotată este suficientă.
- (d) (5 pcte) Ce este un *TLB* (*Translation Look-aside Buffer*) și la ce folosește? Care sunt implicațiile context-switching-ului asupra funcționării TLB și ce soluții se pot folosi?

3. (20 pte) Gestiunea proceselor/thread-urilor.

- (a) (4 pte) Enumerați principalele stări în care se poate afla un proces în execuție și explicați în ce condiții se face tranziția între aceste stări? O diagramă adnotată corespunzător e suficientă.
- (b) (4 pte) Care e semantica apelului sistem Unix **vfork** și de ce era necesar? Care sunt limitările de utilizare (sau precauțiile necesare) atunci când se folosește **vfork**?
- (c) (4 pte) Ce sunt modelele de multithreading? Dați două exemple și explicați pe scurt cum funcționează fiecare dintre ele.
- (d) (4 pte) De ce nu e corectă soluția lui Peterson în cazul procesoarelor moderne? Sugerați o soluție pentru a remedia problema.
- (e) (4 pte) Care sunt condițiile necesare producerii deadlock-ului?

4. (10 pte) Sisteme de stocare a datelor și fișiere.

- (a) (2 pte) Ce este un *i-node* Unix și la ce folosește? Explicați răspunsul pe scurt.
- (b) (2 pte) Ce este *Virtual Filesystem Switch (VFS)* și la ce folosește? Explicați răspunsul pe scurt.
- (c) (3 pte) Care dintre operațiile de lucru cu fișiere nu există în cazul lucrului cu directoare? De ce nu există? Explicați răspunsul pe scurt.
- (d) (3 pte) Care sunt componentele principale ale timpului de acces la date pentru discuri dure organizate pe cilindri, piste și sectoare? Detaliați răspunsul pe scurt.

5. (10 pte) Gestiunea memoriei.

Un PC are 128 MB de memorie RAM și folosește un microprocesor care implementează un spațiu de adrese pe 45 de biți, are pagini de 16 KB și folosește PTE-uri (intrări în tabela de pagini) de 8 octeți. Un program care rulează pe acest computer are 10 KB de text și date încărcate la baza spațiului de adrese și 5 KB de stivă la capătul superior al spațiului de adrese.

- (a) (4 pte) Câtă memorie ocupă o tabelă inversată de pagini care mapează programul în RAM?
- (b) (6 pte) Ce probleme apar dacă încercați să folosiți tabela de pagini de la punctul anterior într-un sistem de memorie virtuală bazat pe paginare la cerere? Puteți rezolva aceste probleme? Cu ce cost (de câtă memorie adițională este nevoie)?

6. (6 pte) Algoritmi de planificare a accesului la disc.

Un disc are 40 de cilindri (de la 0 la 39). Fie o secvență de citiri care implică cilindrii 9, 1, 34, 12, 36, 11 și 16, în această ordine. **Comparați** timpii de căutare (*seek time*) ai următorilor algoritmi: **SSTF**, **C-LOOK**. Puteți presupune că la început capul de citire a discului se află în dreptul primului cilindru din lista de cereri.

7. (14 pte) Planificarea proceselor.

(a) O aplicație multimedia conține trei thread-uri de timp real: Tv afișează stream-ul video iar Ts și Td reprezintă canalele stâng respectiv drept ale sursei de sunet stereo. Caracteristicile de timp în milisecunde ale acestor thread-uri sunt:

Thread	Perioada	Timp de calcul
Tv	24	6
Ts	8	3
Td	8	3

(i) (4 pte) Se pot planifica aceste thread-uri conform algoritmului Rate Monotonic? Puteți considera ca $\ln 2 = 0,69$. Explicați răspunsul.

(ii) (3 pte) Se pot planifica aceste thread-uri folosind algoritmul *Least Slack First (LSF)*? (*LSF planifică pentru execuție taskul cu cel mai mic slack, adică durata maximă de timp cu care un task poate fi întârziat fără să piardă deadline-ul*). Explicați răspunsul.

(b) (7 pte) Dată fiind o colecție de procese cu caracteristicile de mai jos, **trasați o diagramă Gantt** pentru o politică de planificare de tip *Multilevel Feedback Queue (MFQ)* ca mai jos și **calculați timpul ei mediu de așteptare**. Timpii de sosire/rulare sunt exprimați în ms, valorile mici exprimă priorități mari iar cuanta de timp a sistemului este de 1 ms. Un proces nu poate pierde procesorul decât la sfârșitul cuantei de timp alocate. Planificarea nu folosește tehnica de aging.

Toate procesele încep în coada de prioritate maximă care este guvernată de o politică de tip **Shortest Remaining Time First (SRTF)** cu priorități și maximum 2 cuante de timp alocate pentru rulare. Când procesele din coada SRTF și-au epuizat timpul de rulare în coada respectivă, ele retrogradează următoarea coada gestionată după o politică **Round Robin (RR)** cu priorități și maximum 1 cuantă de timp de rulare. Când procesele din coada RR și-au epuizat și ele timpul de rulare în coada respectivă, ele sunt retrogradează în ultima coada gestionată după o politică de planificare **First Come First Served (FCFS)**.

Proces	Timp de sosire	Timp de rulare	Prioritate
P1	0	6	2
P2	2	4	1
P3	2	2	3
P4	4	1	1
P5	0	5	2

8. (20 pte) Controlul proceselor. (a) (15 pte) Spalatoria auto Shiny Car are N stații de spălare, fiecare putând spăla o singură mașină la un moment dat. Mașinile așteaptă la coadă până când există cel puțin o stație de spălare liberă. Dacă se eliberează S stații ($1 < S \leq N$), ele sunt ocupate de S mașini dintre cele care așteaptă la coadă. O mașină care intră în curtea spălătoriei ocupă una dintre stațiile de spălare libere, comanda pornirea spălării, așteaptă terminarea spălării după care pleacă din spălătorie. O stație liberă semnalează acest lucru cu lumina verde și așteaptă sosirea unei mașini. Când are o mașină de spălat, stația spală mașina, iar la sfârșitul spălării ridică o barieră și permite mașinii să plece, după care reia ciclul de operare.

Scrieți pseudo-cod semaforizat pentru a modela funcționarea mașinilor, respectiv a stațiilor de spălare, a.î. sa nu apară race conditions (folosiți denumirile de la curs, **down** și **up**, pentru cele două operații cu semafoare descrise în manual sub numele de wait și signal).

Pentru a modela operarea fiecărui tip de comportament, adăugați la codul de mai jos:

- declarațiile de semafoare necesare și valorile lor inițiale;
- alte declarații de variabile necesare modelării problemei
- cod semaforizat pentru rutina masina() executată de către mașini;
- cod semaforizat pentru rutina statie_de_spalare(int j) executată de stația de spălare j;

```
sem_t ____ = ____; // semafoare atâtea cate sunt necesare plus valorile lor inițiale
int N;
// alte declarații de variabile necesare
```

```
masina() {
    // așteaptă sa existe cel puțin o stație de spălare libera
    // alege una dintre stațiile libere, sa zicem k
    k = masina_intra_la_spalat();
    // comanda începerea spălării în stația k
    // așteaptă terminarea spălării
    pleaca_din_spalatorie();
    // mașina a plecat
}

statie_de_spalare(int j) {
    while(1) {
        // semnalează ca stația j e libera
        // așteaptă o mașina
        spala_masina();
        // eliberează mașina si marchează stația de spălare j libera
    }
}
```

(b) (5 pte) Care este rezultatul execuției următorului program? Explicați răspunsul.

```
void call_vfork() {
    if(!vfork())
        return;
}

int main(int argc, char *argv[]) {
    call_vfork();
    exit(0);
}
```